Optional

Carsten Gips (HSBI)

TL;DR

Häufig hat man in Methoden den Fall, dass es keinen Wert gibt, und man liefert dann `null` als "kein Wert vorhanden" zurück. Dies führt dazu, dass die Aufrufer eine entsprechende `null`-Prüfung für die Rückgabewerte durchführen müssen, bevor sie das Ergebnis nutzen können.

`Optional` schließt elegant den Fall "kein Wert vorhanden" ein: Es kann mit der Methode `Optional.ofNullable()` das Argument in ein Optional verpacken (Argument != `null`) oder ein `Optional.empty()` zurückliefern ("leeres" Optional, wenn Argument == `null`).

Man kann Optionals prüfen mit `isEmpty()` und `ifPresent()` und dann direkt mit `ifPresent()`, `orElse()` und `orElseThrow()` auf den verpackten Wert zugreifen. Besser ist aber der Zugriff über die Stream-API von `Optional`: `map()`, `filter`, `flatMap()`, ... Dabei gibt es keine terminalen Operationen - es handelt sich ja auch nicht um einen Stream, nur die Optik erinnert daran.

`Optional` ist vor allem für Rückgabewerte gedacht, die den Fall "kein Wert vorhanden" einschließen sollen. Attribute, Parameter und Sammlungen sollten nicht `Optional`-Referenzen speichern, sondern "richtige" (unverpackte) Werte (und eben zur Not `null`). `Optional` ist kein Ersatz für `null`-Prüfung von Methoden-Parametern (nutzen Sie hier beispielsweise passende Annotationen). `Optional` ist auch kein Ersatz für vernünftiges Exception-Handling im Fall, dass etwas Unerwartetes passiert ist. Liefern Sie **niemals** `null` zurück, wenn der Rückgabetyp der Methode ein `Optional` ist!

Videos

- VL Optional
- Demo Optional
- Demo Optional: Beispiel aus der Praxis im PM-Dungeon

Unterlagen

- PDF-Version

---

1 Motivation

```java
public class LSF {
    private Set<Studi> sl;

    public Studi getBestStudi() {
        if (sl == null) return null;  // Fehler: Es gibt noch keine Sammlung

        Studi best = null;
        for (Studi s : sl) {
            if (best == null) best = s;
            if (best.credits() < s.credits()) best = s;
        }
        return best;
    }
}

public static void main(String... args) {
    LSF lsf = new LSF();

```

```
19      Studi best = lsf.getBestStudi();
20      if (best != null) {
21          String name = best.name();
22          if (name != null) {
23              // mach was mit dem Namen ...
24          }
25      }
26  }
```

1.1 Problem: `null` wird an (zu) vielen Stellen genutzt

- Es gibt keinen Wert ("not found")
- Felder wurden (noch) nicht initialisiert
- Es ist ein Problem oder etwas Unerwartetes aufgetreten

=> Parameter und Rückgabewerte müssen stets auf **`null`** geprüft werden (oder Annotationen wie `@NotNull` eingesetzt werden ...)

1.2 Lösung

- `Optional<T>` für Rückgabewerte, die "kein Wert vorhanden" mit einschließen (statt **`null`** bei Abwesenheit von Werten)
- `@NotNull`/`@Nullable` für Parameter einsetzen (oder separate Prüfung)
- Exceptions werfen in Fällen, wo ein Problem aufgetreten ist

1.3 Anmerkungen

- Verwendung von **`null`** auf Attribut-Ebene (Klassen-interne Verwendung) ist okay!
- `Optional<T>` ist **kein** Ersatz für **`null`**-Checks!
- **`null`** ist **kein** Ersatz für vernünftiges Error-Handling! Das häufig zu beobachtende "Irgendwas Unerwartetes ist passiert, hier ist **`null`**" ist ein **Anti-Pattern**!

1.4 Beispiel aus der Praxis im PM-Dungeon

Schauen Sie sich einmal das Review zu den `ecs.components.ai.AITools` in https://github.com/Dungeon-CampusMinden/Dungeon/pull/128#pullrequestreview-1254025874 an.

133 game/src/ecs/components/ai/AITools.java Viewed

```java
+
+    /**
+     * Finds the path to a random (accessible) tile in the given radius, starting from the position
+     * of the given entity.
+     *
+     * @param entity Entity whose position is the center point
+     * @param radius Search radius
+     * @return Path from the position of the entity to the randomly selected tile
+     */
+    public static GraphPath<Tile> calculateNewPath(Entity entity, float radius) {
+        PositionComponent pc = (PositionComponent) entity.getComponent(PositionComponent.name);
+        VelocityComponent vc = (VelocityComponent) entity.getComponent(VelocityComponent.name);
+        if (pc != null && vc != null) {
+            ILevel level = ECS.currentLevel;
+            Point position = pc.getPosition();
+            List<Tile> tiles = new ArrayList<>();
+            for (float x = position.x - radius; x <= position.x + radius; x++) {
+                for (float y = position.y - radius; y <= position.y + radius; y++) {
+                    tiles.add(level.getTileAt(new Point(x, y).toCoordinate()));
+                }
+            }
+            tiles.removeIf(Objects::isNull);
+            tiles.removeIf(tile -> !tile.isAccessible());
+            Coordinate newPosition = tiles.get(random.nextInt(tiles.size())).getCoordinate();
+            return level.findPath(
+                    level.getTileAt(position.toCoordinate()), level.getTileAt(newPosition));
+        }
+        return null;
```

Die Methode `AITools#calculateNewPath` soll in der Umgebung einer als Parameter übergebenen Entität nach einem Feld (`Tile`) suchen, welches für die Entität betretbar ist und einen Pfad von der Position der Entität zu diesem Feld an den Aufrufer zurückliefern.

Zunächst wird in der Entität nach einer `PositionComponent` und einer `VelocityComponent` gesucht. Wenn es (eine) diese(r) Components nicht in der Entität gibt, wird der Wert **`null`** an den Aufrufer von `AITools#calculateNewPath` zurückgeliefert. (*Anmerkung*: Interessanterweise wird in der Methode nicht mit der `VelocityComponent` gearbeitet.)

Dann wird in der `PositionComponent` die Position der Entität im aktuellen Level abgerufen. In einer Schleife werden alle Felder im gegebenen Radius in eine Liste gespeichert. (*Anmerkung*: Da dies über die **`float`**-Werte passiert und nicht über die Feld-Indizes wird ein `Tile` u.U. recht oft in der Liste abgelegt. Können Sie sich hier einfache Verbesserungen überlegen?)

Da `level.getTileAt()` offenbar als Antwort auch **`null`** zurückliefern kann, werden nun zunächst per `tiles.removeIf(Objects::isNull);` all diese **`null`**-Werte wieder aus der Liste entfernt. Danach erfolgt die Prüfung, ob die verbleibenden Felder betretbar sind und nicht-betretbare Felder werden entfernt.

Aus den verbleibenden (betretbaren) Feldern in der Liste wird nun eines zufällig ausgewählt und per `level.findPath()` ein Pfad von der Position der Entität zu diesem Feld berechnet und zurückgeliefert. (*Anmerkung*: Hier wird ein zufälliges Tile in der Liste der umgebenden Felder gewählt, von diesem die Koordinaten bestimmt, und dann noch einmal aus dem Level das dazugehörige Feld geholt - dabei hatte man die Referenz auf das Feld bereits in der Liste. Können Sie sich hier eine einfache Verbesserung überlegen?)

Zusammengefasst:

- Die als Parameter `entity` übergebene Referenz darf offenbar *nicht* **`null`** sein. Die ersten beiden Statements in der Methode rufen auf dieser Referenz Methoden auf, was bei einer **`null`**-Referenz zu einer `NullPointer`-Exception führen würde. Hier wäre **`null`** ein Fehlerzustand.

- `entity.getComponent()` kann offenbar **`null`** zurückliefern, wenn die gesuchte Component nicht vorhanden ist. Hier wird **`null`** als “kein Wert vorhanden” genutzt, was dann nachfolgende **`null`**-Checks notwendig macht.
- Wenn es die gewünschten Components nicht gibt, wird dem Aufrufer der Methode **`null`** zurückgeliefert. Hier ist nicht ganz klar, ob das einfach nur “kein Wert vorhanden” ist oder eigentlich ein Fehlerzustand?
- `level.getTileAt()` kann offenbar **`null`** zurückliefern, wenn kein Feld an der Position vorhanden ist. Hier wird **`null`** wieder als “kein Wert vorhanden” genutzt, was dann nachfolgende **`null`**-Checks notwendig macht (Entfernen aller **`null`**-Referenzen aus der Liste).
- `level.findPath()` kann auch wieder **`null`** zurückliefern, wenn kein Pfad berechnet werden konnte. Hier ist wieder nicht ganz klar, ob das einfach nur “kein Wert vorhanden” ist oder eigentlich ein Fehlerzustand? Man könnte beispielsweise in diesem Fall ein anderes Feld probieren?

Der Aufrufer bekommt also eine `NullPointer`-Exception, wenn der übergebene Parameter `entity` nicht vorhanden ist oder den Wert **`null`**, wenn in der Methode etwas schief lief oder schlicht kein Pfad berechnet werden konnte oder tatsächlich einen Pfad. Damit wird der Aufrufer gezwungen, den Rückgabewert vor der Verwendung zu untersuchen.

Allein in dieser einen kurzen Methode macht `null` so viele extra Prüfungen notwendig und den Code dadurch schwerer lesbar und fehleranfälliger! `null` wird als (unvollständige) Initialisierung und als Rückgabewert und für den Fehlerfall genutzt, zusätzlich ist die Semantik von `null` nicht immer klar. (*Anmerkung*: Der Gebrauch von **`null`** hat nicht wirklich etwas mit “der Natur eines ECS” zu tun. Die Methode wurde mittlerweile komplett überarbeitet und ist in der hier gezeigten Form glücklicherweise nicht mehr zu finden.)

Entsprechend hat sich in diesem Review die nachfolgende Diskussion ergeben:

133 game/src/ecs/components/ai/AITools.java Viewed

AMatutat marked this conversation as resolved. Hide resolved

cagix on Jan 18 • edited Member

Was bedeutet `null` als Ergebnis? Die Operation hat nicht geklappt, weil es keine `PositionComponent` und/oder `VelocityComponent` gab? Wäre das nicht die Stelle, eine passende Exception auszulösen (dito in `PositionComponent` und `VelocityComponent`)? Dann braucht der Aufrufer nicht die ganzen `if (wuppie != null) { ...} else {return null;}` ... (das "Pattern" taucht ganz schön häufig auf)

AMatutat on Jan 18 Member Author

Jop. Das pattern gibt es häufiger und die Problematik lässt sich auf die Natur eines ECS zurückführen. Ich wollte nochmal in der großen Welt gucken, wie andere damit umgehen.

Dauer Lösung ist es so nicht.
Ist ja auch nur draft

cagix on Jan 18 Member

Schon klar. Leider hält ein Provisorium oft länger, als einem lieb ist ;)

Ich denke, die Lösung heißt `Exception` oder `Option<T>`, wobei letzteres einen etwas anderen Code-Stil nach sich zieht (oder Du endest dann analog zu den `if (null)`-Abfragen mit `.isEmpty()`).

cagix on Jan 18 • edited Member

Schau mal https://github.com/Programmiermethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#lsf-liefert-jetzt-optional-zurück und https://github.com/Programmiermethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#zugriff-auf-optional-objekte: das passt eigentlich exakt zu dem Problem hier, der Zugriff dann mit `best = lsf.getBestStudi().orElseThrow();` (mit `public Optional<Studi> getBestStudi()`).

AMatutat on Jan 18 • edited Member Author

Das wäre eine Möglichkeit, welches sich am aktuellen Konzept orientiert.

Ich weiß nur nicht, ob ein null-Check (in welcher Form auch immer) überhaupt notwendig ist.
Vielleicht sollte das Programm hier crashen.

Immerhin hab ich (als Anwender) in meiner Entität eine Funktion hinterlegt (über das `AIComponent`), welches nunmal das `PositionComponent` und `VelocityComponent` voraussetzt, dann bin ich vielleicht auch dafür verantwortlich, dass ich die benötigten Components auch hinzufügen.

Aber so wirklich richtig klingt das auch nicht.

Man könnte auch an diesen stellen ein `if(positionComponent==null) new PositionComponent ...` machen, aber das wirkt auf mich "übergriffig".

133 game/src/ecs/components/ai/AITools.java Viewed

aber das wirkt auf mich "übergriffig".

cagix on Jan 19 • edited Member

- Ich bin immer ein Fan davon, möglichst robusten Code zu schreiben und vorhersehbare Probleme zu berücksichtigen.
- Fehler des Anwenders kann ich nur bedingt auffangen - ein Crash scheint in Deinem Szenario unvermeidlich.
- Ein Crash darf aber informativ sein und den Nutzer in die Lage versetzen zu erkennen, was er falsch gemacht hat bzw. was von ihm erwartet wird - wenn es hilfreiche Fehlermeldungen von uns gibt.
- Ein Teil des Problems sind aber wir selbst, wenn wir "null" als Initialisierung und/oder Rückgabewerte überhaupt erlauben. Das ist zwar bequem, führt aber zu den ganzen Problemen bzw. unterstützt diese. Und es gibt ja moderne Mittel und Wege, vernünftiger damit umzugehen.

cagix on Jan 19 Member

Ich glaube, Du willst wirklich durchgängig mit Option arbeiten.

Und wenn etwas nicht da ist (aber da sein sollte), dann ist das imho ein Fehlerzustand. Und die Antwort in Java auf Fehler ist nicht null, sondern eine Exception.

AMatutat on Jan 19 • edited Member Author

> Und wenn etwas nicht da ist (aber da sein sollte), dann ist das imho ein Fehlerzustand. Und die Antwort in Java auf Fehler ist nicht null, sondern eine Exception.

Oder es ist ein Fehler im System-Design. Das ist die andere Möglichkeit.

Wenn wir null weiter "erlauben", dann ist Optional the way to go.

Hab das Mal ausgelagert #131

1

cagix on Jan 19 Member

ja, das zieht sich ja durch das ganze design und muss gut überlegt werden

2 Erzeugen von *Optional*-Objekten

Konstruktor ist `private` ...

- "Kein Wert": `Optional.empty()`
- Verpacken eines non-`null` Elements: `Optional.of()`
  (`NullPointerException` wenn Argument `null`!)

- Verpacken eines "unsicheren"/beliebigen Elements: `Optional.ofNullable()`
  - Liefert verpacktes Element, oder
  - `Optional.empty()`, falls Element `null` war

Es sollte in der Praxis eigentlich nur wenige Fälle geben, wo ein Aufruf von `Optional.of()` sinnvoll ist. Ebenso ist `Optional.empty()` nur selten sinnvoll.

Stattdessen sollte stets `Optional.ofNullable()` verwendet werden.

`null` kann nicht nicht in `Optional<T>` verpackt werden! (Das wäre dann eben `Optional.empty()`.)

3 LSF liefert jetzt *Optional* zurück

```java
public class LSF {
    private Set<Studi> sl;

    public Optional<Studi> getBestStudi() throws NullPointerException {
        // Fehler: Es gibt noch keine Sammlung
        if (sl == null) throw new NullPointerException("There ain't any collection");

        Studi best = null;
        for (Studi s : sl) {
            if (best == null) best = s;
            if (best.credits() < s.credits()) best = s;
        }

        // Entweder Optional.empty() (wenn best==null) oder Optional.of(best) sonst
        return Optional.ofNullable(best);
    }
}
```

Das Beispiel soll verdeutlichen, dass man im Fehlerfall nicht einfach `null` oder `Optional.empty()` zurückliefern soll, sondern eine passende Exception werfen soll.

Wenn die Liste aber leer ist, stellt dies keinen Fehler dar! Es handelt sich um den Fall "kein Wert vorhanden". In diesem Fall wird statt `null` nun ein `Optional.empty()` zurückgeliefert, also ein Objekt, auf dem der Aufrufer die üblichen Methoden aufrufen kann.

4 Zugriff auf *Optional*-Objekte

In der funktionalen Programmierung gibt es schon lange das Konzept von `Optional`, in Haskell ist dies beispielsweise die Monade `Maybe`. Allerdings ist die Einbettung in die Sprache von vornherein mit berücksichtigt worden, insbesondere kann man hier sehr gut mit *Pattern Matching* in der Funktionsdefinition auf den verpackten Inhalt reagieren.

In Java gibt es die Methode `Optional#isEmpty()`, die einen Boolean zurückliefert und prüft, ob es sich um ein leeres `Optional` handelt oder ob hier ein Wert "verpackt" ist.

Für den direkten Zugriff auf die Werte gibt es die Methoden `Optional#orElseThrow()` und `Optional#orElse()`. Damit kann man auf den verpackten Wert zugreifen, oder es wird eine Exception geworfen bzw. ein Ersatzwert geliefert.

Zusätzlich gibt es `Optional#isPresent()`, die als Parameter ein `java.util.function.Consumer` erwartet, also ein funktionales Interface mit einer Methode `void accept(T)`, die das Objekt verarbeitet.

```java
Studi best;

// Testen und dann verwenden
if (!lsf.getBestStudi().isEmpty()) {
```

```
5       best = lsf.getBestStudi().get();
6       // mach was mit dem Studi ...
7   }
8
9   // Arbeite mit Consumer
10  lsf.getBestStudi().ifPresent(studi -> {
11      // mach was mit dem Studi ...
12  });
13
14  // Studi oder Alternative (wenn Optional.empty())
15  best = lsf.getBestStudi().orElse(anne);
16
17  // Studi oder NoSuchElementException (wenn Optional.empty())
18  best = lsf.getBestStudi().orElseThrow();
```

Es gibt noch eine Methode `get()`, die so verhält wie `orElseThrow()`. Da man diese Methode vom Namen her schnell mit einem Getter verwechselt, ist sie mittlerweile *deprecated*.

Anmerkung: Da `getBestStudi()` eine `NullPointerException` werfen kann, sollte der Aufruf möglicherweise in ein **`try`**/**`catch`** verpackt werden. Dito für `orElseThrow()`.

Beispiel: optional.traditional.Demo

5 Einsatz mit Stream-API

```
1   public class LSF {
2       ...
3       public Optional<Studi> getBestStudi() throws NullPointerException {
4           if (sl == null) throw new NullPointerException("There ain't any
                collection");
5           return sl.stream()
6                   .sorted((s1, s2) -> s2.credits() - s1.credits())
7                   .findFirst();
8       }
9   }
10
11
12  public static void main(String... args) {
13      ...
14      String name = lsf.getBestStudi()
15                       .map(Studi::name)
16                       .orElseThrow();
17  }
```

Beispiel: optional.streams.Demo

Im Beispiel wird in `getBestStudi()` die Sammlung als Stream betrachtet, über die Methode `sorted()` und den Lamda-Ausdruck für den `Comparator` sortiert ("falsch" herum: absteigend in den Credits der Studis in der Sammlung), und `findFirst()` ist die terminale Operation auf dem Stream, die ein `Optional<Studi>` zurückliefert: entweder den Studi mit den meisten Credits (verpackt in `Optional<Studi>`) oder `Optional.empty()`, wenn es überhaupt keine Studis in der Sammlung gab.

In `main()` wird dieses `Optional<Studi>` mit den Stream-Methoden von `Optional<T>` bearbeitet, zunächst mit `Optional#map()`. Man braucht nicht selbst prüfen, ob das von `getBestStudi()` erhaltene Objekt leer ist oder nicht, da dies von `Optional#map()` erledigt wird: Es wendet die Methodenreferenz auf den verpackten Wert an (sofern dieser vorhanden ist) und liefert damit den Namen des Studis als `Optional<String>` verpackt zurück. Wenn es keinen Wert, also nur `Optional.empty()` von `getBestStudi()` gab, dann ist der Rückgabewert von `Optional#map()` ein `Optional.empty()`. Wenn der Name, also der Rückgabewert von `Studi::name`, **`null`** war, dann wird ebenfalls ein `Optional.empty()` zurückgeliefert. Dadurch wirft `orElseThrow()` dann eine `NoSuchElementException`. Man kann also direkt mit dem String `name` weiterarbeiten ohne extra **`null`**-Prüfung - allerdings will man noch ein Exception-Handling einbauen (dies fehlt im obigen Beispiel aus Gründen der Übersicht) …

6 Weitere *Optionals*

Für die drei primitiven Datentypen **`int`**, **`long`** und **`double`** gibt es passende Wrapper-Klassen von `Optional<T>`: `OptionalInt`, `OptionalLong` und `OptionalDouble`.

Diese verhalten sich analog zu `Optional<T>`, haben aber keine Methode `ofNullable()`, da dies hier keinen Sinn ergeben würde: Die drei primitiven Datentypen repräsentieren Werte - diese können nicht **`null`** sein.

7 Regeln für *Optional*

1. Nutze `Optional` nur als Rückgabe für “kein Wert vorhanden”

   `Optional` ist nicht als Ersatz für eine **`null`**-Prüfung o.ä. gedacht, sondern als Repräsentation, um auch ein “kein Wert vorhanden” zurückliefern zu können.

2. Nutze nie **`null`** für eine `Optional`-Variable oder einen `Optional`-Rückgabewert

   Wenn man ein `Optional` als Rückgabe bekommt, sollte das niemals selbst eine **`null`**-Referenz sein. Das macht das gesamte Konzept kaputt!

   Nutzen Sie stattdessen `Optional.empty()`.

3. Nutze `Optional.ofNullable()` zum Erzeugen eines `Optional`

   Diese Methode verhält sich “freundlich” und erzeugt automatisch ein `Optional.empty()`, wenn das Argument **`null`** ist. Es gibt also keinen Grund, dies mit einer Fallunterscheidung selbst erledigen zu wollen.

   Bevorzugen Sie `Optional.ofNullable()` vor einer manuellen Fallunterscheidung und dem entsprechenden Einsatz von `Optional.of()` und `Optional.empty()`.

4. Erzeuge keine `Optional` als Ersatz für die Prüfung auf **`null`**

   Wenn Sie auf **`null`** prüfen müssen, müssen Sie auf **`null`** prüfen. Der ersatzweise Einsatz von `Optional` macht es nur komplexer - prüfen müssen Sie hinterher ja immer noch.

5. Nutze `Optional` nicht in Attributen, Methoden-Parametern und Sammlungen

   Nutzen Sie `Optional` vor allem für Rückgabewerte.

   Attribute sollten immer direkt einen Wert haben oder **`null`**, analog Parameter von Methoden o.ä. … Hier hilft `Optional` nicht, Sie müssten ja trotzdem eine **`null`**-Prüfung machen, nur eben dann über den `Optional`, wodurch dies komplexer und schlechter lesbar wird.

   Aus einem ähnlichen Grund sollten Sie auch in Sammlungen keine `Optional` speichern!

6. Vermeide den direkten Zugriff (`ifPresent()`, `orElseThrow()` …)

   Der direkte Zugriff auf ein `Optional` entspricht dem Prüfen auf **`null`** und dann dem Auspacken. Dies ist nicht nur Overhead, sondern auch schlechter lesbar.

   Vermeiden Sie den direkten Zugriff und nutzen Sie `Optional` mit den Stream-Methoden. So ist dies von den Designern gedacht.

8 Interessante Links

- "Using Optionals"
- "What You Might Not Know About Optional"
- "Experienced Developers Use These 7 Java Optional Tips to Remove Code Clutter"
- "Code Smells: Null"
- "Class Optional"

9 Wrap-Up

`Optional` als Rückgabe für "kein Wert vorhanden"

- `Optional.ofNullable()`: Erzeugen eines `Optional`
  - Entweder Objekt "verpackt" (Argument != **`null`**)
  - Oder `Optional.empty()` (Argument == **`null`**)
- Prüfen mit `isEmpty()` und `ifPresent()`
- Direkter Zugriff mit `ifPresent()`, `orElse()` und `orElseThrow()`
- Stream-API: `map()`, `filter()`, `flatMap()`, …

- Attribute, Parameter und Sammlungen: nicht `Optional` nutzen
- Kein Ersatz für **`null`**-Prüfung!

Schöne Doku: "Using Optionals".

10 Zum Nachlesen

- Oracle Corporation (2025)
- Ullenboom (2021, Kap. 12.6)

Lernziele

- k2: Ich kann erklären, warum Optionals vor allem für Rückgabewerte gedacht sind
- k2: Ich kann erklären, warum kein null zurückgeliefert werden darf, wenn der Rückgabetyp ein Optional<T> ist
- k3: Ich kann (ggf. leere) Optionals mit Optional.ofNullable() erzeugen
- k3: Ich kann auf Optionals klassisch über die direkten Hilfsmethoden der Klasse zugreifen
- k3: Ich kann auf Optionals elegant per Stream-API zugreifen

Challenges

Optional und Stream-API

1. Erklären Sie den folgenden Code-Schnipsel aus dem Dungeon:

```
Skill fireball =
    new Skill(
        new FireballSkill(
            () ->
                hero.fetch(CollideComponent.class)
                    .map(cc -> cc.center(hero).add(viewDirection.
                        toPoint()))
                    .orElseThrow(
                        () -> MissingComponentException.build(hero,
                            CollideComponent.class)),
            FIREBALL_RANGE,
            FIREBALL_SPEED,
            FIREBALL_DMG),
        1);
```

   Hinweis: `Entity#fetch: <T extends Component> Optional<T> fetch(final Class<T> klass)`.

2. Was würde sich ändern, wenn statt `map` ein `flatMap` verwendet würde? Wie ist das bei richtigen Streams?
3. Was passiert im folgenden Beispiel? Warum funktioniert das auch ohne terminale Stream-Operation?

```
Game.hero()
    .flatMap(e -> e.fetch(AmmunitionComponent.class))
    .map(AmmunitionComponent::resetCurrentAmmunition);
```

   Hinweis: `Game.hero(): static Optional<Entity> hero()`.

4. Können Sie die beiden obigen Beispiele in "klassischer" Schreibweise umformulieren?

String-Handling

Können Sie den folgenden Code so umschreiben, dass Sie statt der `if`-Abfragen und der einzelnen direkten Methodenaufrufe die Stream-API und `Optional<T>` nutzen?

```
String format(final String text, String replacement) {
    if (text.isEmpty()) {
        return "";
    }

    final String trimmed = text.trim();
    final String withSpacesReplaced = trimmed.replaceAll(" +", replacement
        );

    return replacement + withSpacesReplaced + replacement;
}
```

Ein Aufruf `format("Hello World ... ", "_");` liefert den String "`_Hello_World_..._`".

Quellen

Oracle Corporation. 2025. „Learn Java“. 2025. https://dev.java/learn/.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ecee47d (lecture: add code snippet to challenges (Optional), 2025-05-05)